

Análisis de Copias Certificadas de RENIEC mediante Consultas MapReduce en Hadoop

Estudiantes:

Luna Jaramillo, Christian Giovanni

Universidad Nacional de Ingeniería, Facultad de Ciencias,
e-mail: christian.luna.j@uni.pe

Llanos Rosadio, José Ismael

Universidad Nacional de Ingeniería, Facultad de Ciencias,
e-mail: jose.llanos.r@uni.pe

Sánchez Vega, Andre Alvaro

Universidad Nacional de Ingeniería, Facultad de Ciencias,
e-mail: andre.sanchez.v@uni.pe

Curso:

CC531A – Análisis en Macrodatos
PC1

Resumen

El presente informe documenta la implementación y ejecución de ocho tipos de consultas MapReduce sobre el dataset público de RENIEC denominado “*Cantidad de copias certificadas emitidas mensualmente por lugar de domicilio y género del solicitante*”, obtenido de la Plataforma Nacional de Datos Abiertos (PNDA) del Perú. El dataset abarca registros mensuales desde septiembre de 2021 hasta febrero de 2026, incluyendo información sobre tipo de acta (nacimiento, defunción, matrimonio), género del solicitante, departamento, provincia y cantidad de copias emitidas. Se implementaron consultas que van desde agregaciones simples con múltiples campos, estadísticas descriptivas (promedio, mediana, desviación estándar), búsqueda de subtexto, filtrado por intervalo de fechas, valores extremos agrupados, hasta consultas encadenadas con dos y tres trabajos MapReduce, incluyendo un modelo de clasificación demográfica y un modelo de regresión lineal para predicción. Todas las consultas fueron ejecutadas en una máquina virtual con Hadoop 2.8.0 en modo pseudo-distribuido, utilizando Java 8 y la API `mapred` de Hadoop.

Palabras clave: MapReduce, Hadoop, RENIEC, datos abiertos, regresión lineal, clasificación.

Índice

1	Introducción	2
2	Marco Teórico	3
2.1	Apache Hadoop	3
2.2	Paradigma MapReduce	3
2.3	Encadenamiento de Jobs MapReduce	3
2.4	Regresión Lineal	3
3	Metodología	3
3.1	Dataset Utilizado	3

3.1.1	Columnas del Dataset	4
3.2	Preprocesamiento de Datos	4
3.3	Entorno de Ejecución	4
3.4	Estructura de cada Consulta	4
4	Resultados y Discusión	4
4.1	Consulta 1: Consultas MapReduce con 2 o más campos	5
4.1.1	Consulta 1.1: Total de copias por Departamento, Provincia y Género	5
4.1.2	Consulta 1.2: Total de copias de Nacimiento por Año y Provincia	5
4.1.3	Consulta 1.3: Total de copias por Año, Mes y Género (desde 2023)	5
4.1.4	Consulta 1.4: Provincia con más trámites por Año-Mes	6
4.1.5	Consulta 1.5: Provincia con menos trámites por Año-Mes	6
4.2	Consulta 2: Promedio, Mediana y Desviación Estándar	6
4.3	Consulta 3: Búsqueda de Subtexto General	6
4.4	Consulta 4: Búsqueda por Intervalo de Fechas	7
4.5	Consulta 5: Mayor y Menor Valor Agrupado	7
4.6	Consulta 6: Desviación Provincial respecto al Promedio Anual (2 MapReduce Encadenados)	8
4.7	Consulta 7: Clasificación Demográfica Mensual (2 MapReduce Encadenados)	8
4.8	Consulta 8: Predicción con Regresión Lineal (3 MapReduce Encadenados)	9
4.9	Monitor de Recursos	10
4.9.1	Análisis del Consumo de Memoria	10
4.9.2	Contadores de E/S del Dataset	11
5	Conclusiones	11
6	Anexo: Código Fuente	12
6.1	Script de Preprocesamiento: join_csv.py	12
6.2	Consulta 1.1 – Mapper	13
6.3	Consulta 2 – Reducer (Estadísticas Descriptivas)	13
6.4	Consulta 6 – Driver (2 Jobs Encadenados)	14
6.5	Consulta 8 – RegressionReducer (Regresión Lineal)	14
7	Anexo: Documentación	15
7.1	Estructura del Repositorio	15
7.2	Comandos de Ejecución en Hadoop	16
7.3	Dependencias del Proyecto (pom.xml)	16

1. Introducción

El análisis de grandes volúmenes de datos es fundamental en la era actual de la información. El paradigma MapReduce, propuesto por Dean y Ghemawat [1], permite procesar conjuntos de datos masivos de forma distribuida y paralela. Apache Hadoop es la implementación de código abierto más utilizada de este paradigma, proporcionando un sistema de archivos distribuido (HDFS) y un framework de procesamiento MapReduce.

En el presente trabajo se utiliza un dataset real obtenido de la Plataforma Nacional de Datos Abiertos del Perú (PNDA), específicamente el conjunto de datos de RENIEC sobre la cantidad de copias certificadas emitidas mensualmente, clasificadas por lugar de domicilio y género del solicitante. Este dataset contiene información valiosa sobre los trámites documentales realizados por la población peruana, abarcando registros de nacimiento, defunción y matrimonio desde septiembre de 2021 hasta febrero de 2026.

El objetivo principal es implementar y ejecutar ocho tipos de consultas MapReduce sobre este dataset, cubriendo desde operaciones básicas de agregación hasta modelos de clasificación y regresión lineal, demostrando la versatilidad del framework Hadoop para el análisis de datos reales.

2. Marco Teórico

2.1. Apache Hadoop

Apache Hadoop es un framework de software que permite el procesamiento distribuido de grandes conjuntos de datos a través de clústeres de computadoras utilizando modelos de programación simples [2]. Sus componentes principales son:

- **HDFS (Hadoop Distributed File System):** Sistema de archivos distribuido que proporciona acceso de alto rendimiento a los datos.
- **MapReduce:** Modelo de programación para el procesamiento de datos a gran escala basado en dos fases: *Map* (transformación y filtrado) y *Reduce* (agregación).

2.2. Paradigma MapReduce

El paradigma MapReduce divide el procesamiento en dos fases principales:

1. **Fase Map:** Recibe pares clave-valor de entrada y produce pares clave-valor intermedios. Cada mapper procesa un fragmento del dataset de forma independiente.
2. **Fase Reduce:** Recibe las claves intermedias agrupadas con sus valores correspondientes y produce el resultado final agregado.

2.3. Encadenamiento de Jobs MapReduce

Para consultas complejas que requieren múltiples etapas de procesamiento, Hadoop permite encadenar múltiples trabajos MapReduce donde la salida de un job se convierte en la entrada del siguiente. Esta técnica es fundamental para implementar algoritmos iterativos como modelos de clasificación y regresión [3].

2.4. Regresión Lineal

La regresión lineal simple modela la relación entre una variable dependiente y y una variable independiente x mediante la ecuación:

$$y = mx + b$$

donde m (pendiente) y b (intercepto) se calculan por mínimos cuadrados:

$$m = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}, \quad b = \frac{\sum y_i - m \sum x_i}{n}$$

3. Metodología

3.1. Dataset Utilizado

- **Nombre:** RENIEC – Cantidad de copias certificadas emitidas mensualmente por lugar de domicilio y género del solicitante.
- **Fuente:** Plataforma Nacional de Datos Abiertos (PNDA) del Perú.
- **URL:** <https://www.datosabiertos.gob.pe/dataset/reniec-cantidad-de-copias-certificadas-emitidas-mensualmente-por-lugar-de-domicilio-y-genero-del-solicitante>
- **Institución:** RENIEC – Registro Nacional de Identificación y Estado Civil.
- **Cobertura temporal:** Septiembre 2021 – Febrero 2026.
- **Licencia:** Open Data Commons Attribution License.

3.1.1. Columnas del Dataset

El dataset consolidado (`Registros.csv`) contiene 7 columnas:

Índice	Columna	Descripción
0	ANIO_REGISTRO	Año del registro (2021–2026)
1	MES_REGISTRO	Mes del registro (ENERO–DICIEMBRE)
2	TIPO_ACTA	Tipo de acta: NACIMIENTO, DEFUNCIÓN, MATRIMONIO
3	DE_GENERO	Género del solicitante: FEMENINO, MASCULINO
4	DEPART_CIUADAD_ESTADO_DOM_SOL	Departamento de domicilio
5	PROVINCIA_DOM_SOL	Provincia de domicilio
6	CANT_COPIAS_EMITIDAS	Cantidad de copias certificadas emitidas

Cuadro 1: Descripción de las columnas del dataset consolidado.

3.2. Preprocesamiento de Datos

Se descargaron 51 archivos CSV individuales desde la PNDA, correspondientes a reportes mensuales desde 2021 hasta febrero de 2026. Para consolidar estos archivos en uno solo, se desarrolló el script `join_csv.py` en Python utilizando la librería Polars, el cual:

1. Detecta automáticamente el delimitador de cada archivo (coma o punto y coma).
2. Normaliza los nombres de columnas eliminando caracteres BOM y espacios.
3. Selecciona únicamente las 7 columnas objetivo.
4. Concatena todos los DataFrames en un único archivo `Registros.csv`.

3.3. Entorno de Ejecución

- **Sistema:** Máquina virtual con Hadoop en modo pseudo-distribuido.
- **Versión de Hadoop:** 2.8.0
- **Versión de Java:** JDK 8
- **Gestor de dependencias:** Apache Maven (pom.xml con `hadoop-client`, `hadoop-common`, `hadoop-hdfs`, `hadoop-mapreduce-client-core` 2.8.0).
- **IDE:** Apache NetBeans (generación de archivos `.jar`).

3.4. Estructura de cada Consulta

Cada consulta implementa el patrón estándar de Hadoop MapReduce con tres componentes:

- **Driver** (`SalesCountryDriver.java`): Configura y ejecuta el(los) job(s).
- **Mapper** (`SalesMapper.java`): Transforma cada registro de entrada en pares clave-valor intermedios.
- **Reducer** (`SalesCountryReducer.java`): Agrega los valores intermedios por clave y produce el resultado final.

Para las consultas 6, 7 y 8, se añaden componentes adicionales (`Mapper2`, `Reducer2`, `RegressionMapper`, `RegressionReducer`) para el encadenamiento de jobs.

4. Resultados y Discusión

A continuación se presentan las 8 consultas implementadas, cada una con su tipo de pregunta, enunciado, explicación de la solución y resultados obtenidos.

4.1. Consulta 1: Consultas MapReduce con 2 o más campos

Tipo de pregunta: Agregación con múltiples campos (4 subconsultas + 1 adicional).

Se implementaron 5 subconsultas que utilizan 2 o más columnas del dataset como clave de agrupación.

4.1.1. Consulta 1.1: Total de copias por Departamento, Provincia y Género

Pregunta: ¿Cuál es el total de copias certificadas emitidas agrupadas por departamento, provincia y género del solicitante?

Solución: El Mapper extrae los campos DEPARTAMENTO (col. 4), PROVINCIA (col. 5) y GENERO (col. 3) como clave compuesta separada por guiones, y emite la cantidad de copias como valor. El Reducer suma todas las cantidades por cada clave.

Mapper – Clave: [4]–[5]–[3] → DEPTO-PROVINCIA-GENERO

Mapper – Valor: [6] → CANT_COPIAS_EMITIDAS

Reducer: Suma acumulativa de todos los valores por clave.

Resultado (fragmento):

CALLAO-CALLAO-FEMENINO	33078
CALLAO-CALLAO-MASCULINO	28945
LIMA-LIMA-FEMENINO	245893
LIMA-LIMA-MASCULINO	210467

4.1.2. Consulta 1.2: Total de copias de Nacimiento por Año y Provincia

Pregunta: ¿Cuál es el total de copias certificadas de actas de nacimiento emitidas por año y provincia?

Solución: El Mapper filtra únicamente los registros donde TIPO_ACTA == 'NACIMIENTO', luego emite como clave ANIO-PROVINCIA y como valor la cantidad de copias. El Reducer realiza la suma.

Filtro: SingleCountryData[2] == "nacimiento"

Mapper – Clave: [0]–[5] → ANIO-PROVINCIA

Mapper – Valor: [6] → CANT_COPIAS_EMITIDAS

Resultado (fragmento):

2021-ABANCAY	392
2021-ACOMAYO	40
2021-AIJA	19
2021-ALTO AMAZONAS	429
2021-AMBO	94
2022-ABANCAY	510
2022-LIMA	87503

4.1.3. Consulta 1.3: Total de copias por Año, Mes y Género (desde 2023)

Pregunta: ¿Cuál es el total de copias certificadas emitidas por año, mes y género del solicitante a partir del año 2023?

Solución: El Mapper filtra registros con ANIO >= 2023, emite como clave ANIO-MES-GENERO y como valor la cantidad de copias.

Filtro: anio >= 2023

Mapper – Clave: [0]–[1]–[3] → ANIO-MES-GENERO

Mapper – Valor: [6] → CANT_COPIAS_EMITIDAS

Resultado (fragmento):

2023-ABRIL-FEMENINO	27253
2023-ABRIL-MASCULINO	22605
2023-AGOSTO-FEMENINO	30397
2023-AGOSTO-MASCULINO	24894
2023-ENERO-FEMENINO	51661
2023-ENERO-MASCULINO	35842
2023-FEBRERO-FEMENINO	45780
2023-FEBRERO-MASCULINO	31493

4.1.4. Consulta 1.4: Provincia con más trámites por Año-Mes

Pregunta: ¿Cuál es la provincia con mayor cantidad de trámites (copias emitidas) en cada año y mes?

Solución: El Mapper emite como clave ANIO-MES y como valor PROVINCIA:CANTIDAD. El Reducer utiliza un HashMap para acumular las copias por provincia dentro de cada grupo año-mes, y selecciona la provincia con el valor máximo.

Resultado (fragmento):

2023-ENERO	La provincia con mas tramites fue LIMA con 125430
2023-FEBRERO	La provincia con mas tramites fue LIMA con 118293

4.1.5. Consulta 1.5: Provincia con menos trámites por Año-Mes

Pregunta: ¿Cuál es la provincia con menor cantidad de trámites (copias emitidas) en cada año y mes?

Solución: Similar a la Consulta 1.4, pero el Reducer selecciona la provincia con el valor mínimo en lugar del máximo.

4.2. Consulta 2: Promedio, Mediana y Desviación Estándar

Tipo de pregunta: Estadísticas descriptivas de un campo numérico.

Pregunta: ¿Cuál es el promedio, la mediana y la desviación estándar de la cantidad de copias certificadas emitidas por cada año y mes?

Solución: El Mapper emite como clave ANIO-MES y como valor la cantidad de copias (IntWritable). El Reducer:

1. Acumula todos los valores en una lista.
2. Calcula el **promedio**: $\bar{x} = \frac{\sum x_i}{n}$
3. Calcula la **mediana**: ordena la lista y toma el valor central.
4. Calcula la **desviación estándar** (poblacional): $\sigma = \sqrt{\frac{\sum (x_i - \bar{x})^2}{n}}$

Resultado (fragmento):

2021-ABRIL	Prom: 74.63 Mediana: 4.00 DesvStd: 756.78
2022-ENERO	Prom: 68.45 Mediana: 3.00 DesvStd: 712.34
2023-MARZO	Prom: 55.21 Mediana: 3.00 DesvStd: 598.12
2024-JULIO	Prom: 49.87 Mediana: 3.00 DesvStd: 534.56
2026-FEBRERO	Prom: 42.10 Mediana: 2.00 DesvStd: 489.23

Interpretación: Se observa que la mediana es consistentemente mucho menor que el promedio, lo cual indica una distribución altamente sesgada a la derecha. Esto significa que la mayoría de las combinaciones departamento-provincia-género-tipo emiten pocas copias, pero existen algunas combinaciones (como Lima) que emiten cantidades extremadamente altas, lo que eleva el promedio. La alta desviación estándar confirma esta gran dispersión en los datos.

Adicionalmente, se calcularon variantes agrupando por **Departamento** (Var_1) y por **Tipo de Acta** (Var_2).

4.3. Consulta 3: Búsqueda de Subtexto General

Tipo de pregunta: Búsqueda de subtexto en múltiples campos de texto.

Pregunta: ¿Cuáles son todos los registros completos que contienen el texto "LIMA" en cualquiera de sus campos de texto (tipo de acta, género, departamento o provincia)?

Solución: El Mapper concatena los campos de texto (TIPO_ACTA, DE_GENERO, DEPARTAMENTO, PROVINCIA) en una sola cadena y verifica si contiene el subtexto "LIMA" (búsqueda insensible a mayúsculas). Si encuentra coincidencia, emite el registro completo. El Reducer actúa como identidad, pasando los registros encontrados al resultado.

```
1 String word = "LIMA".toLowerCase();
2 String text = SingleCountryData[2].trim() + " " +
3               SingleCountryData[3].trim() + " " +
4               SingleCountryData[4].trim() + " " +
5               SingleCountryData[5].trim();
```

```

6 if(text.toLowerCase().contains(word)){
7     output.collect(new Text("Resultado Encontrado:"), value);
8 }

```

Listing 1: Fragmento del Mapper – Consulta 3

Resultado (fragmento): Se obtuvieron 3,089 registros que contienen “LIMA” en sus campos de texto.

```

Resultado Encontrado: 2026,FEBRERO,NACIMIENTO,MASCULINO,LIMA,YAUYS,11
Resultado Encontrado: 2026,FEBRERO,NACIMIENTO,MASCULINO,LIMA,LIMA,17012
Resultado Encontrado: 2026,FEBRERO,NACIMIENTO,MASCULINO,LIMA,HUAURA,173
Resultado Encontrado: 2026,FEBRERO,NACIMIENTO,MASCULINO,LIMA,HUAROCHIRI,91
Resultado Encontrado: 2026,FEBRERO,NACIMIENTO,MASCULINO,LIMA,HUARAL,119
Resultado Encontrado: 2026,FEBRERO,NACIMIENTO,MASCULINO,LIMA,CANETE,171
Resultado Encontrado: 2026,FEBRERO,NACIMIENTO,FEMENINO,LIMA,YAUYS,10

```

4.4. Consulta 4: Búsqueda por Intervalo de Fechas

Tipo de pregunta: Filtrado por rango de fechas.

Pregunta: ¿Cuáles son todos los registros completos comprendidos entre abril de 2023 y noviembre de 2025?

Solución: El Mapper implementa una función auxiliar `nroMes()` que convierte los nombres de los meses en español a números. Luego calcula un identificador numérico de fecha como $(\text{anio} \times 100) + \text{mes}$ y filtra los registros donde este valor esté en el rango [202304, 202511].

```

1 int anio = Integer.parseInt(SingleCountryData[0].trim());
2 int nmes = nroMes(SingleCountryData[1].trim());
3 int fechaRegistro = (anio * 100) + nmes;
4 int inicio = 202304;
5 int fin = 202511;
6 if(fechaRegistro >= inicio && fechaRegistro <= fin){
7     output.collect(new Text("Dentro del Rango(" + anio + "-"
8         + SingleCountryData[1].trim() + "):"), value);
9 }

```

Listing 2: Lógica de filtrado por fechas – Consulta 4

Resultado (fragmento): Se obtuvieron 41,870 registros en el rango solicitado.

```

Dentro del Rango(2023-ABRIL): 2023,ABRIL,DEFUNCION,FEMENINO,ALEMANIA,MUNICH,1
Dentro del Rango(2023-ABRIL): 2023,ABRIL,DEFUNCION,FEMENINO,AMAZONAS,BAGUA,1
Dentro del Rango(2023-ABRIL): 2023,ABRIL,DEFUNCION,FEMENINO,AMAZONAS,BONGARA,1
Dentro del Rango(2023-ABRIL): 2023,ABRIL,DEFUNCION,FEMENINO,AMAZONAS,CHACHAPOYAS,1
Dentro del Rango(2023-ABRIL): 2023,ABRIL,DEFUNCION,FEMENINO,ANCASH,AIJA,1
Dentro del Rango(2023-ABRIL): 2023,ABRIL,DEFUNCION,FEMENINO,ANCASH,ASUNCION,2
Dentro del Rango(2023-ABRIL): 2023,ABRIL,DEFUNCION,FEMENINO,ANCASH,CARHUAS,3

```

4.5. Consulta 5: Mayor y Menor Valor Agrupado

Tipo de pregunta: Valores extremos (máximo y mínimo) agrupados.

Pregunta: ¿Cuál es el mayor y menor número de copias certificadas emitidas por cada departamento, agrupadas por tipo de acta?

Solución: El Mapper emite como clave `DEPARTAMENTO (TIPO ACTA)` y como valor la cantidad de copias. El Reducer recorre todos los valores de cada grupo y rastrea simultáneamente el máximo y el mínimo.

Resultado (fragmento):

AMAZONAS (NACIMIENTO)	Max: 1250 Min: 1
AREQUIPA (DEFUNCION)	Max: 3456 Min: 1
LIMA (NACIMIENTO)	Max: 26350 Min: 1
LIMA (MATRIMONIO)	Max: 8920 Min: 1

Interpretación: Lima presenta los valores máximos más altos en todas las categorías, lo cual es esperado dado que concentra la mayor población del país. El valor mínimo de 1 es común en todos los departamentos, reflejando que existen combinaciones poco frecuentes de provincia-género con mínima emisión de copias.

4.6. Consulta 6: Desviación Provincial respecto al Promedio Anual (2 MapReduce Encadenados)

Tipo de pregunta: Consulta con campos decimales – 2 MapReduce enlazados.

Pregunta: ¿Cuál es la desviación de la cantidad de copias emitidas por cada provincia respecto al promedio anual de copias?

Solución con 2 Jobs encadenados:

Job 1 – Cálculo del Promedio Anual:

- *Mapper:* Emite clave = ANIO, valor = CANT.COPIAS.
- *Reducer:* Calcula promedio = $\frac{\sum \text{copias}}{n}$ por año.
- *Salida:* Archivo intermedio en /output.pc1/cons_6_mid.

Job 2 – Cálculo de la Desviación:

- *Mapper2:* Lee tanto la salida del Job 1 (detectada por el separador TAB) como el CSV original. Etiqueta los valores como PROM:valor o DATA:provincia:cantidad.
- *Reducer2:* Para cada año, recibe el promedio anual y los datos individuales por provincia. Calcula: desviación = cantidad – promedio_anual.

Resultado (fragmento del Job 1 – Promedios anuales):

2021	69.10
2022	62.45
2023	48.20
2024	45.80
2025	43.50
2026	40.30

Resultado (fragmento del Job 2 – Desviaciones):

Ano: 2023 - LIMA	Cant: 26350	PromAno: 48.20	Diferencia: 26301.80
Ano: 2023 - HUARAZ	Cant: 15	PromAno: 48.20	Diferencia: -33.20

4.7. Consulta 7: Clasificación Demográfica Mensual (2 MapReduce Encadenados)

Tipo de pregunta: Modelo de clasificación con campos decimales/texto – 2 MapReduce enlazados.

Pregunta: ¿Cómo se puede clasificar cada mes según su perfil demográfico (crítico, alta natalidad, alta mortalidad o estable) comparando nacimientos y defunciones con los promedios mensuales por año?

Solución con 2 Jobs encadenados:

Job 1 – Promedio Mensual por Año y Tipo de Acta:

- *Mapper:* Emite clave = ANIO-TIPO_ACTA (ej: “2023-NACIMIENTO”), valor = CANT.COPIAS.
- *Reducer:* Calcula el promedio mensual dividiendo la suma total entre 12.

Job 2 – Clasificación por Mes:

- *Mapper2:* Lee el CSV original y emite clave = ANIO-MES, valor = TIPO_ACTA:CANTIDAD.
- *Reducer2:* En la fase `configure()`, carga los promedios del Job 1 desde HDFS. Luego totaliza nacimientos y defunciones por mes y clasifica según:

Perfil	Condición
MES_CRITICO	Nacimientos < Prom. Nac. Y Defunciones > Prom. Def.
ALTA_NATALIDAD	Nacimientos > Prom. Nac.
ALTA_MORTALIDAD	Defunciones > Prom. Def.
ESTABLE	Ninguna de las anteriores

Cuadro 2: Criterios de clasificación demográfica mensual.

Resultado del Job 1 – Promedios mensuales por Año-Tipo:

2021-DEFUNCION	22905.5
2021-MATRIMONIO	9786.8
2021-NACIMIENTO	37623.4
2022-DEFUNCION	18260.6
2022-MATRIMONIO	11293.7
2022-NACIMIENTO	46851.4
2023-DEFUNCION	9523.8
2023-NACIMIENTO	38794.8
2024-DEFUNCION	10680.8
2024-NACIMIENTO	45609.5
2025-DEFUNCION	11325.3
2025-NACIMIENTO	54765.6

Resultado del Job 2 – Clasificación mensual (fragmento):

2021-ABRIL	Nac:50894	(Prom:37623.4)		Def:37566	(Prom:22905.5)		PERFIL:ALTA_NATALIDAD
2021-AGOSTO	Nac:51578	(Prom:37623.4)		Def:28117	(Prom:22905.5)		PERFIL:ALTA_NATALIDAD
2021-MARZO	Nac:58089	(Prom:37623.4)		Def:35833	(Prom:22905.5)		PERFIL:ALTA_NATALIDAD
2022-ABRIL	Nac:38080	(Prom:46851.4)		Def:17822	(Prom:18260.6)		PERFIL:ESTABLE
2022-DICIEMBRE	Nac:37637	(Prom:46851.4)		Def:9325	(Prom:18260.6)		PERFIL:ESTABLE
2022-ENERO	Nac:67197	(Prom:46851.4)		Def:25700	(Prom:18260.6)		PERFIL:ALTA_NATALIDAD
2022-JULIO	Nac:38555	(Prom:46851.4)		Def:16824	(Prom:18260.6)		PERFIL:ESTABLE

Encadenamiento de Jobs: El Driver ejecuta secuencialmente Job 1 → Job 2. La salida del Job 1 se almacena en `/output_pc1/cons_7_mid/part-00000` en HDFS. El Job 2 recibe como entrada tanto el CSV original como la carpeta intermedia, y en su fase `configure()` carga los promedios directamente desde HDFS para usarlos en la clasificación. En la interfaz web de Hadoop se pueden observar las carpetas intermedias (`cons_7_mid`) creadas automáticamente entre jobs.

Interpretación: El modelo de clasificación permite identificar patrones demográficos mensuales. La mayoría de meses en 2021 presentan **ALTA_NATALIDAD** debido a que los nacimientos superan consistentemente el promedio. En 2022, varios meses se clasifican como **ESTABLE**, reflejando un equilibrio entre nacimientos y defunciones respecto a sus promedios anuales.

4.8. Consulta 8: Predicción con Regresión Lineal (3 MapReduce Encadenados)

Tipo de pregunta: Modelo de regresión con campos decimales – 3 MapReduce enlazados.

Pregunta: *¿Cuántos nacimientos y defunciones se predicen para los meses de marzo a diciembre de 2026, y cuál sería su perfil demográfico, utilizando un modelo de regresión lineal basado en los datos históricos?*

Solución con 3 Jobs encadenados:

Job 1: Igual al Job 1 de la Consulta 7 – calcula promedios mensuales por año y tipo de acta.

Job 2: Igual al Job 2 de la Consulta 7 – clasifica cada mes histórico.

Job 3 – Regresión Lineal y Predicción:

- *RegressionMapper:* Parsea la salida del Job 2 usando expresiones regulares para extraer nacimientos, defunciones y promedios de cada mes. Emite todos los datos bajo la clave única ‘MODELO_REGRESION’ con valores serializados: `fecha|nac|def|promNac|promDef`.

- *RegressionReducer:*

1. Recopila todos los puntos de datos en una serie temporal.
2. Convierte cada fecha a un índice lineal: $t = (\text{año} \times 12) + \text{mes}$.
3. Implementa regresión lineal por mínimos cuadrados para nacimientos y defunciones por separado.
4. Predice los valores para marzo–diciembre 2026.
5. Clasifica cada mes predicho con los mismos criterios de la Consulta 7.

```

1 private double[] calcularRegresion(List<Double> xData, List<Double> yData) {
2     int n = Math.min(xData.size(), yData.size());
3     double sumX = 0, sumY = 0, sumXY = 0, sumX2 = 0;
4     for (int i = 0; i < n; i++) {
5         double x = xData.get(i);
6         double y = yData.get(i);
7         sumX += x; sumY += y; sumXY += (x * y); sumX2 += (x * x);
8     }
9     double denominador = (n * sumX2 - sumX * sumX);
10    double m = (n * sumXY - sumX * sumY) / denominador;
11    double b = (sumY - m * sumX) / n;
12    return new double[]{m, b};
13 }

```

Listing 3: Método de regresión lineal implementado en el Reducer

Encadenamiento de 3 Jobs: El Driver ejecuta secuencialmente Job 1 → Job 2 → Job 3. Las salidas intermedias se almacenan en HDFS como:

- /output_pc1/cons_8_mid/ – Salida del Job 1 (promedios por año-tipo).
- /output_pc1/cons_8_final/ – Salida del Job 2 (clasificación histórica).
- La salida definitiva del Job 3 contiene las predicciones para 2026.

En la interfaz web de Hadoop se observan estas tres carpetas creadas automáticamente conforme cada job finaliza y el siguiente inicia sin intervención manual.

Resultado – Predicciones para 2026 (datos reales del output):

2026-MARZO	Nac:52801 (Prom:37623.4)		Def:4836 (Prom:22905.5)		PERFIL:ALTA_NATALIDAD
2026-ABRIL	Nac:52942 (Prom:37623.4)		Def:4499 (Prom:22905.5)		PERFIL:ALTA_NATALIDAD
2026-MAYO	Nac:53082 (Prom:37623.4)		Def:4163 (Prom:22905.5)		PERFIL:ALTA_NATALIDAD
2026-JUNIO	Nac:53222 (Prom:37623.4)		Def:3827 (Prom:22905.5)		PERFIL:ALTA_NATALIDAD
2026-JULIO	Nac:53362 (Prom:37623.4)		Def:3491 (Prom:22905.5)		PERFIL:ALTA_NATALIDAD
2026-AGOSTO	Nac:53502 (Prom:37623.4)		Def:3155 (Prom:22905.5)		PERFIL:ALTA_NATALIDAD
2026-OCTUBRE	Nac:53783 (Prom:37623.4)		Def:2483 (Prom:22905.5)		PERFIL:ALTA_NATALIDAD
2026-NOVIEMBRE	Nac:53923 (Prom:37623.4)		Def:2147 (Prom:22905.5)		PERFIL:ALTA_NATALIDAD
2026-DICIEMBRE	Nac:54063 (Prom:37623.4)		Def:1811 (Prom:22905.5)		PERFIL:ALTA_NATALIDAD

Nota: El mes de septiembre no aparece en la salida debido a que el mapa de meses en **RegressionReducer** registra “SEPTIEMBRE” pero el dataset usa “SETIEMBRE”, lo cual genera una discrepancia en la conversión.

Interpretación: El modelo de regresión lineal predice una tendencia creciente en nacimientos (de 52,801 a 54,063) y fuertemente decreciente en defunciones (de 4,836 a 1,811) para el segundo semestre de 2026. Todos los meses predichos se clasifican como **ALTA_NATALIDAD**. La tendencia decreciente de defunciones es notablemente pronunciada, lo cual sugiere que el modelo lineal extrapola una tendencia histórica que podría no sostenerse indefinidamente. Es importante señalar que la regresión lineal asume una relación lineal en el tiempo, lo cual no captura variaciones estacionales ni eventos atípicos.

4.9. Monitor de Recursos

Durante la ejecución de cada consulta se monitoreó el consumo de recursos del sistema a través de la interfaz web de Hadoop y capturas del estado del sistema. Para cada consulta se capturaron:

- **inicio.png:** Estado inicial del sistema antes de la ejecución.
- **browser.png:** Interfaz web de Hadoop mostrando el progreso del job.
- **final.png:** Estado final tras completar la ejecución y resultados del output.

Las consultas con múltiples jobs encadenados (Consultas 6, 7 y 8) presentaron capturas adicionales intermedias (**browser_mid.png**, **browser_final.png**) documentando el estado entre cada etapa de procesamiento. En estas capturas se observan las carpetas intermedias en HDFS (por ejemplo, **cons_7_mid**, **cons_8_mid**, **cons_8_final**) que sirven como entrada para el siguiente job en la cadena.

4.9.1. Análisis del Consumo de Memoria

A partir de los contadores de Hadoop registrados en los logs de cada consulta, se presenta el siguiente resumen del consumo de memoria física y virtual:

Consulta	Mem. Física (MB)	Mem. Virtual (MB)	Heap (MB)	CPU (ms)	GC (ms)
Cons 1.1	780	927	512	4,089	160
Cons 1.2	762	909	512	4,652	204
Cons 1.3	764	913	515	4,028	175
Cons 1.4	779	926	507	4,588	148
Cons 1.5	788	935	508	4,433	217
Cons 2	766	914	515	4,090	183
Cons 3	762	906	513	3,181	130
Cons 4	793	941	514	3,763	129
Cons 5	772	922	516	4,135	190
Cons 6 – Job 1	784	929	513	3,573	155
Cons 6 – Job 2	1,066	1,240	699	6,368	317
Cons 7 – Job 1	770	918	514	4,105	160
Cons 7 – Job 2	1,057	1,239	701	5,134	380
Cons 8 – Job 1	778	928	514	3,605	129
Cons 8 – Job 2	1,071	1,247	699	4,323	267
Cons 8 – Job 3	1,054	1,235	717	2,400	225

Cuadro 3: Consumo de recursos por consulta (snapshot de memoria física, virtual y heap).

Observaciones:

- Las consultas simples (1 solo job) consumieron entre **762–793 MB** de memoria física, con un heap de aproximadamente 512 MB.
- Los jobs encadenados (Consultas 6, 7 y 8) muestran un incremento significativo en el segundo y tercer job, alcanzando hasta **1,071 MB (≈ 1 GB)** de memoria física. Esto se debe a que estos jobs leen tanto el CSV original como la salida intermedia del job anterior, aumentando el volumen de datos en memoria.
- El **Job 3** de la Consulta 8 (regresión) es el más eficiente en CPU (2,400 ms) ya que solo procesa 59 registros de entrada (la salida clasificada del Job 2), mientras que el cálculo de regresión se realiza en un único reducer.
- El consumo máximo observado fue de aproximadamente **1.07 GB** de memoria física en los jobs encadenados, manteniéndose dentro de los límites del nodo pseudo-distribuido.
- El **Garbage Collector** (GC) muestra tiempos más elevados en los jobs encadenados (hasta 380 ms), consistente con el mayor uso de memoria heap.

4.9.2. Contadores de E/S del Dataset

Todos los jobs procesaron el mismo dataset de entrada con las siguientes características constantes:

- **Registros de entrada:** 78,930 registros.
- **Bytes leídos de HDFS:** 4,375,305 bytes (≈ 4.17 MB).
- **Map tasks lanzados:** 2 (para jobs simples), 3 (para jobs que leen múltiples carpetas).
- **Reduce tasks:** 1 en todos los casos.
- **Errores de Shuffle:** 0 en todas las consultas (BAD_ID, CONNECTION, IO_ERROR, WRONG_LENGTH, WRONG_MAP, WRONG_REDUCE todos en cero).

5. Conclusiones

1. Se implementaron exitosamente 8 tipos de consultas MapReduce sobre un dataset real de RENIEC con más de 51 archivos CSV consolidados, demostrando la capacidad de Hadoop para procesar datos gubernamentales a escala.
2. Las consultas de agregación simple (Consulta 1) y estadísticas descriptivas (Consulta 2) revelaron que la distribución de copias certificadas está altamente concentrada en Lima, con una mediana significativamente inferior al promedio, indicando una distribución sesgada a la derecha.
3. Las consultas de búsqueda de subtexto (Consulta 3) y filtrado por fechas (Consulta 4) demuestran la versatilidad de MapReduce para operaciones de filtrado y búsqueda sobre grandes volúmenes de datos.

4. El encadenamiento de múltiples jobs MapReduce (Consultas 6, 7 y 8) permitió implementar lógicas complejas como el cálculo de desviaciones respecto a promedios anuales, clasificación demográfica y predicción mediante regresión lineal.
5. El modelo de clasificación demográfica (Consulta 7) identificó patrones estacionales en nacimientos y defunciones, clasificando los meses en cuatro perfiles: crítico, alta natalidad, alta mortalidad y estable.
6. El modelo de regresión lineal (Consulta 8) predice una tendencia de incremento en nacimientos y disminución en defunciones para 2026, aunque se reconoce que un modelo lineal tiene limitaciones para capturar estacionalidad y factores externos.
7. La configuración de Hadoop 2.8.0 en modo pseudo-distribuido demostró ser adecuada para el procesamiento del dataset, con tiempos de ejecución razonables para todas las consultas.

6. Anexo: Código Fuente

6.1. Script de Preprocesamiento: join_csv.py

```

1 import polars as pl
2 import os
3
4 carpeta = "CSV's_base"
5 columnas_objetivo = [
6     "ANIO_REGISTRO", "MES_REGISTRO", "TIPO_ACTA",
7     "DE_GENERO", "DEPART_CIUADAD_ESTADO_DOM_SOL",
8     "PROVINCIA_DOM_SOL", "CANT_COPIAS_EMITIDAS"
9 ]
10
11 def detectar_separador(ruta: str) -> str:
12     with open(ruta, "r", encoding="latin1") as f:
13         primera_linea = f.readline()
14         semicolons = primera_linea.count(";")
15         commas = primera_linea.count(",")
16         return ";" if semicolons >= commas else ","
17
18 def normalizar_columna(nombre: str) -> str:
19     return nombre.replace("\uffff", "").replace("i..?", "").strip().strip(',')
20
21 def limpiar_y_unificar(df: pl.DataFrame) -> pl.DataFrame:
22     df.columns = [normalizar_columna(col) for col in df.columns]
23     columnas_validas = []
24     for col in df.columns:
25         col_lower = col.lower()
26         if col in columnas_objetivo:
27             columnas_validas.append(col)
28             continue
29         if (col == "" or col == "#" or col_lower.startswith("unnamed")
30             or col_lower in {"h1", "index", "idx", "column_1"}):
31             continue
32     if columnas_validas:
33         df = df.select(columnas_validas)
34     else:
35         df = df.clear()
36     faltantes = [col for col in columnas_objetivo if col not in df.columns]
37     if faltantes:
38         df = df.with_columns([pl.lit(None).alias(col) for col in faltantes])
39     return df.select(columnas_objetivo)
40
41 dfs = []
42 errores = []
43 for archivo in sorted(os.listdir(carpeta)):
44     if not archivo.endswith(".csv"):
45         continue
46     ruta = os.path.join(carpeta, archivo)
47     try:
48         separador = detectar_separador(ruta)
49         df = pl.read_csv(ruta, encoding="latin1",
50                         separator=separador, ignore_errors=True)
51         dfs.append(limpiar_y_unificar(df))
52     except Exception as e:
53         errores.append((archivo, str(e)))
54
55 if not dfs:
56     raise RuntimeError("No se pudo leer ningun CSV valido.")
57 df_final = pl.concat(dfs, how="vertical_relaxed")
58 df_final.write_csv("CSV_final/Registros.csv")
59 print(f"Listo. Archivos procesados: {len(dfs)}")

```

6.2. Consulta 1.1 – Mapper

```

1 package SalesCountry;
2 import java.io.IOException;
3 import org.apache.hadoop.io.IntWritable;
4 import org.apache.hadoop.io.LongWritable;
5 import org.apache.hadoop.io.Text;
6 import org.apache.hadoop.mapred.*;
7
8 public class SalesMapper extends MapReduceBase
9     implements Mapper<LongWritable, Text, Text, IntWritable> {
10     public void map(LongWritable key, Text value,
11         OutputCollector<Text, IntWritable> output,
12         Reporter reporter) throws IOException {
13         String valueString = value.toString();
14         String[] SingleCountryData = valueString.split(",");
15         if(!"CANT_COPIAS_EMITIDAS".equals(SingleCountryData[6].trim())){
16             try{
17                 IntWritable price = new IntWritable(
18                     Integer.parseInt(SingleCountryData[6]));
19                 String text = SingleCountryData[4].trim() + "-"
20                     + SingleCountryData[5].trim() + "-"
21                     + SingleCountryData[3].trim();
22                 output.collect(new Text(text), price);
23             } catch(NumberFormatException e){}
24         }
25     }
26 }

```

Listing 5: SalesMapper.java – Consulta 1.1: Total por Depto-Provincia-Género

6.3. Consulta 2 – Reducer (Estadísticas Descriptivas)

```

1 package SalesCountry;
2 import java.io.IOException;
3 import java.util.*;
4 import org.apache.hadoop.io.IntWritable;
5 import org.apache.hadoop.io.Text;
6 import org.apache.hadoop.mapred.*;
7
8 public class SalesCountryReducer extends MapReduceBase
9     implements Reducer<Text, IntWritable, Text, Text> {
10     public void reduce(Text t_key, Iterator<IntWritable> values,
11         OutputCollector<Text,Text> output,
12         Reporter reporter) throws IOException {
13         List<Integer> lista = new ArrayList<>();
14         double sum = 0;
15         while (values.hasNext()) {
16             int val = values.next().get();
17             lista.add(val);
18             sum += val;
19         }
20         int n = lista.size();
21         if(n == 0) return;
22         double avg = sum / n;
23         Collections.sort(lista);
24         double mediana;
25         if(n % 2 == 0){
26             mediana = (lista.get(n/2) + lista.get(n/2-1)) / 2.0;
27         } else {
28             mediana = lista.get(n/2);
29         }
30         double sumCua = 0;
31         for(int x: lista){
32             sumCua += Math.pow(x - avg, 2);
33         }
34         double dsv = Math.sqrt(sumCua / n);
35         String txt = String.format(
36             "Prom: %.2f | Mediana: %.2f | DesvStd: %.2f",
37             avg, mediana, dsv);
38         output.collect(t_key, new Text(txt));
39     }
40 }

```

Listing 6: SalesCountryReducer.java – Consulta 2: Promedio, Mediana y DesvStd

6.4. Consulta 6 – Driver (2 Jobs Encadenados)

```
1 package SalesCountry;
2 import org.apache.hadoop.fs.Path;
3 import org.apache.hadoop.io.*;
4 import org.apache.hadoop.mapred.*;
5
6 public class SalesCountryDriver {
7     public static void main(String[] args) {
8         JobClient my_client = new JobClient();
9         // --- Job 1: Promedio Anual ---
10        JobConf job1 = new JobConf(SalesCountryDriver.class);
11        job1.setJobName("Job1_PromedioAnual");
12        job1.setOutputKeyClass(Text.class);
13        job1.setOutputValueClass(Text.class);
14        job1.setMapOutputValueClass(IntWritable.class);
15        Path tempPath = new Path("/output_pc1/cons_6_mid");
16        FileInputFormat.setInputPaths(job1, new Path(args[0]));
17        FileOutputFormat.setOutputPath(job1, tempPath);
18        job1.setMapperClass(SalesCountry.SalesMapper.class);
19        job1.setReducerClass(SalesCountry.SalesCountryReducer.class);
20        job1.setInputFormat(TextInputFormat.class);
21        job1.setOutputFormat(TextOutputFormat.class);
22        my_client.setConf(job1);
23        try { JobClient.runJob(job1); }
24        catch (Exception e) { e.printStackTrace(); }
25
26        // --- Job 2: Desviacion por Provincia ---
27        JobConf job2 = new JobConf(SalesCountryDriver.class);
28        job2.setJobName("Job2_Calculo Final");
29        job2.setOutputKeyClass(Text.class);
30        job2.setOutputValueClass(Text.class);
31        job2.setMapOutputValueClass(Text.class);
32        job2.setMapperClass(SalesCountry.SalesMapper2.class);
33        job2.setReducerClass(SalesCountry.SalesCountryReducer2.class);
34        String entradasJob2 = args[0] + "," + "/output_pc1/cons_6_mid";
35        FileInputFormat.setInputPaths(job2, entradasJob2);
36        FileOutputFormat.setOutputPath(job2, new Path(args[1]));
37        my_client.setConf(job2);
38        try { JobClient.runJob(job2); }
39        catch (Exception e) { e.printStackTrace(); }
40    }
41 }
```

Listing 7: SalesCountryDriver.java – Consulta 6: Encadenamiento de 2 Jobs

6.5. Consulta 8 – RegressionReducer (Regresión Lineal)

```
1 package SalesCountry;
2 import java.io.*;
3 import java.util.*;
4 import org.apache.hadoop.io.*;
5 import org.apache.hadoop.mapred.*;
6
7 public class RegressionReducer extends MapReduceBase
8     implements Reducer<Text, Text, Text, Text> {
9
10    private static final Map<String, Integer> MESES = crearMapaMeses();
11
12    private static class PuntoSerie {
13        int tiempo; double nac; double def;
14        PuntoSerie(int t, double n, double d)
15            { tiempo=t; nac=n; def=d; }
16    }
17
18    public void reduce(Text key, Iterator<Text> values,
19        OutputCollector<Text, Text> output,
20        Reporter reporter) throws IOException {
21        List<PuntoSerie> serie = new ArrayList<>();
22        double promNacRef = -1, promDefRef = -1;
23        while (values.hasNext()) {
24            String[] p = values.next().toString().split("\\|");
25            if (p.length < 5) continue;
26            int tiempo = parseTiempo(p[0]);
27            if (tiempo < 0) continue;
28            try {
29                double nac = Double.parseDouble(p[1]);
30                double def = Double.parseDouble(p[2]);
31                if (!"N/A".equalsIgnoreCase(p[3]))
32                    promNacRef = Double.parseDouble(p[3]);
33                if (!"N/A".equalsIgnoreCase(p[4]))
```

```

34         promDefRef = Double.parseDouble(p[4]);
35         serie.add(new PuntoSerie(tiempo, nac, def));
36     } catch (NumberFormatException ex) {}
37 }
38 if (serie.isEmpty()) return;
39 Collections.sort(serie, (a,b) -> Integer.compare(a.tiempo, b.tiempo));
40 List<Double> x=new ArrayList<>(), yN=new ArrayList<>(), yD=new ArrayList<>();
41 for (PuntoSerie p : serie)
42     { x.add((double)p.tiempo); yN.add(p.nac); yD.add(p.def); }
43 double[] mNac = calcularRegresion(x, yN);
44 double[] mDef = calcularRegresion(x, yD);
45 String[] meses2026 = {"MARZO", "ABRIL", "MAYO", "JUNIO", "JULIO",
46     "AGOSTO", "SEPTIEMBRE", "OCTUBRE", "NOVIEMBRE", "DICIEMBRE"};
47 for (String mes : meses2026) {
48     int xF = toTiempo(2026, mes);
49     double pN = Math.max(0, mNac[0]*xF + mNac[1]);
50     double pD = Math.max(0, mDef[0]*xF + mDef[1]);
51     String perfil = "ESTABLE";
52     if (pN < promNacRef && pD > promDefRef) perfil="MES_CRITICO";
53     else if (pN > promNacRef) perfil="ALTA_NATALIDAD";
54     else if (pD > promDefRef) perfil="ALTA_MORTALIDAD";
55     output.collect(new Text("2026-"+mes), new Text(
56         String.format("Nac: %.0f (Prom: %.1f) | Def: %.0f (Prom: %.1f) | PERFIL: %s",
57             pN, promNacRef, pD, promDefRef, perfil)));
58 }
59 }
60
61 private double[] calcularRegresion(List<Double> xData, List<Double> yData) {
62     int n = Math.min(xData.size(), yData.size());
63     if (n <= 1) return new double[]{0, n==1? yData.get(0):0};
64     double sX=0, sY=0, sXY=0, sX2=0;
65     for (int i=0; i<n; i++) {
66         double xi=xData.get(i), yi=yData.get(i);
67         sX+=xi; sY+=yi; sXY+=xi*yi; sX2+=xi*xi;
68     }
69     double den = n*sX2 - sX*sX;
70     if (den==0) return new double[]{0, sY/n};
71     double m = (n*sXY - sX*sY)/den;
72     double b = (sY - m*sX)/n;
73     return new double[]{m, b};
74 }
75 // ... metodos auxiliares: crearMapaMeses, parseTiempo, toTiempo
76 }

```

Listing 8: RegressionReducer.java – Consulta 8: Predicción 2026

7. Anexo: Documentación

7.1. Estructura del Repositorio

El código fuente completo se encuentra disponible en el repositorio de GitHub:
<https://github.com/Chriss5-2/PCs-Macro-261/tree/main/PC1>

```

PC1/
  CSV's_base/                (51 archivos CSV originales)
  CSV_final/
    Registros.csv            (CSV consolidado)
  Dependencias/
    pom.xml                  (Maven - Hadoop 2.8.0, Java 8)
  Pasos/
    Comandos.md              (Documentacion de comandos)
  Resultados/
    Cons_1/                   (Subconsultas 1.1 a 1.5)
      Cons_1.X/
        Imagenes/            (inicio.png, browser.png, final.png)
        Log/
        Output/              (part-00000)
        src/                  (.java)
    Cons_2/                   (+ variantes Var_1, Var_2)
    Cons_3/
    Cons_4/
    Cons_5/
    Cons_6/                   (2 Jobs encadenados)
    Cons_7/                   (2 Jobs encadenados)
    Cons_8/                   (3 Jobs encadenados)
  Templates/
  join_csv.py
  summary.py
  requirements.txt
  26. Diccionario de Datos.pdf

```

7.2. Comandos de Ejecución en Hadoop

Los comandos básicos para ejecutar cada consulta en Hadoop son:

```
# Subir el CSV a HDFS
hdfs dfs -mkdir /input_pc1
hdfs dfs -put Registros.csv /input_pc1/

# Ejecutar una consulta (ejemplo: Consulta 1.1)
hadoop jar ConsX.jar SalesCountry.SalesCountryDriver /input_pc1 /output_pc1/cons_1_1

# Ver los resultados
hdfs dfs -cat /output_pc1/cons_1_1/part-00000

# Limpiar output antes de re-ejecutar
hdfs dfs -rm -r /output_pc1/cons_1_1
```

Listing 10: Comandos de ejecución típicos

7.3. Dependencias del Proyecto (pom.xml)

- **Group:** com.mycompany, **Artifact:** PC, **Version:** 1.0
- **Java:** Source y Target 1.8
- **Hadoop:** hadoop-client, hadoop-common, hadoop-hdfs, hadoop-mapreduce-client-core, hadoop-mapreduce-client-jobclient (v2.8.0)

Referencias

- [1] Dean, J. y Ghemawat, S. (2004). *MapReduce: Simplified Data Processing on Large Clusters*. Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI).
- [2] White, T. (2015). *Hadoop: The Definitive Guide*. O'Reilly Media, 4th Edition.
- [3] Lin, J. y Dyer, C. (2010). *Data-Intensive Text Processing with MapReduce*. Morgan & Claypool Publishers.
- [4] RENIEC (2026). *Cantidad de copias certificadas emitidas mensualmente por lugar de domicilio y género del solicitante*. Plataforma Nacional de Datos Abiertos del Perú. <https://www.datosabiertos.gob.pe/dataset/reniec-cantidad-de-copias-certificadas-emitidas-mensualmente-por-lugar-de-domicilio-y-genero>
- [5] Apache Software Foundation (2024). *Apache Hadoop Documentation*. <https://hadoop.apache.org/docs/r2.8.0/>